

OpenVLA-OFT · arXiv 2502.19645 · Kim, Finn, Liang (Stanford)

Stakeholder

18+3 · seat 3 · WEIGHT 3 · LONGEST

OFT = speed+success FT patch for OpenVLA
Not a new FM · Not FAST · Not $\pi_{0.5}$ substitute

Oct 26 paper club · VLA Architectures

SPINE: AR discrete bins → parallel continuous L1 chunks

OFT $\neq$ FAST $\neq$ FM · FT recipe / speed+success patch · not $\pi_{0.5}$ on BEHAVIOR

OFT = speed+success FT patch

making OpenVLA viable at 25–50 Hz

(26–43× throughput, LIBERO 97.1%) — modernize OpenVLA ops,

don't substitute for $\pi_{0.5}$ on BEHAVIOR.

OFT = FT recipe

not a new FM

Not FAST

still-AR tokenizer path

Not FM / $\pi_{0.5}$

generative continuous expert

GitHub title: [ECE 605] - OpenVLA-OFT Making Stakeholder

arXiv 2502.19645 · openvla-oft.github.io

TAXONOMY · $\text{OFT} \neq \text{FAST} \neq \text{FM}$

Three parallel improvement paths — do not collapse names

OpenVLA-OFT

FT recipe

Parallel continuous
L1 chunks
K=8 / 25

Decode: 1-pass
Loss: L1
Still 7B OpenVLA

FAST

Tokenizer

DCT→BPE discrete
chunk tokens
still AR

Decode: serial AR
Loss: CE
~750 ms/chunk class

π_0 / $\pi_{0.5}$

FM deploy

Continuous FM
chunks / NFE
OpenPI stack

Decode: denoise
Loss: FM / ϵ
 π_0 ALOHA ~291 Hz

Vanilla OpenVLA = AR discrete bins @ ~4.2 Hz → OFT rewrites output signature

OFT does NOT replace $\pi_{0.5}$ for 2026 BEHAVIOR · $\text{OFT} \neq \text{FAST}$ tokenizer swap

OUTPUT SIGNATURE SHIFT

Vanilla OpenVLA → OpenVLA-OFT

Axis	Vanilla OpenVLA	OFT
Decode	Serial AR discrete bins	Parallel continuous decode
Chunk	K=1	K=8 LIBERO / K=25 ALOHA
Head	256-bin CE	Continuous MLP + L1
Hz	~4.2	~109.7 LIBERO (26×); ALOHA ~77.9 (43×)
LIBERO SR	76.5%	97.1% (π ₀ 94.2%)

OFT+ adds FiLM on SigLIP+DINOv2 (multi-view language) — without FiLM, language → ~chance

SIGNATURE SHIFT DIAGRAM

AR discrete single-step → parallel continuous L1 chunks

VANILLA OpenVLA

img + lang → VLM (causal)

AR next-token × D(=7)

$7 \times \{0..255\}$ bins → dequantize

single EE delta · K=1

~4.2 Hz LIBERO · ~1.8 Hz ALOHA

OFT

OpenVLA-OFT

img[+wrist][+proprio] + lang

bidirectional action queries

one forward → $a \in \mathbb{R}^{K \times D}$

continuous L1 MLP · K=8/25

~109.7 Hz · ALOHA ~77.9 Hz

FAST (sibling)

compress chunk → fewer
AR tokens; still sequential
~750 ms chunk class

π_0 / DP / ACT

continuous chunk experts
FM / diffusion / CVAE+L1
smaller bodies → higher Hz

OFT+

OFT + FiLM on both
vision towers
ALOHA multi-view lang

EXACT OFT RECIPE

Sec V-E / App. A — six ingredients (OFT+ = #6)

1

Parallel decoding

Bidirectional attn over
empty action embeds

2

Action chunking

K=8 LIBERO /
K=25 ALOHA

3

Continuous MLP head

4-layer ReLU;
drop 256-bin LM head

4

L1 on [-1,1]

Mean L1 regression
(train until L1 < 0.01)

5

LoRA r=32

Optional wrist +
proprio projector

6

OFT+: FiLM

Lang $\rightarrow \gamma, \beta$ on SigLIP
+ DINOv2 blocks

BOARD TABLE · Hz / SR

Memorize these cells — Stakeholder tattoo

LIBERO

SR avg	76.5% → 97.1%	+20.6 pp
Hz (A100)	4.2 → 109.7	26×
vs π_0 SR	π_0 94.2%	OFT wins
+wrist+proprio Hz	71.4	still $\gg$ vanilla

ALOHA (OFT+)

Hz	1.8 → 77.9	43×
π_0 Hz still	291.6	JAX faster
Scoop	100 vs π_0 93.3	OFT+ best
Put-in-pot	51 vs π_0 42	~half; hard

Latency (tables)

LIBERO Cont-L1 ~ 0.073 s
ALOHA OFT+ 0.321 s
(π_0 0.086 s)

Abstract caveat

“0.07 ms / 0.321 ms”
conflicts with Table II/III
Prefer seconds

Trainable

LIBERO OFT 279M
ALOHA OFT+ 853M
(FiLM +456M)

Rule: OpenVLA/HF/LoRA ops + high Hz → OFT mandatory; stock AR obsolete for 25-50 Hz

Max wall-clock + OpenPI/JAX → $\pi_0/\pi_{0.5}$ still win latency + pretrain coverage

DECISION TREE · OFT vs $\pi_{0.5}$ vs FAST

Stakeholder HIGH priority — say this aloud

Need high-Hz control on OpenVLA stack?

NO

Stock OpenVLA OK for
<10 Hz language demos

YES → adopt OFT

parallel + chunk + Cont-L1

Choose OFT when

- Prismatic / HF / LoRA ops
- Simple L1 FT preferred
- LIBERO-style + moderate demos
- 7B AR-lineage ablations
- Wrist+proprio match robot I/O

Choose $\pi_{0.5}$ when

- OpenPI standardized
- Long-horizon open-world
- 2026 BEHAVIOR primary
- Need multimodal / recovery
- Max wall-clock (291 Hz class)

Choose FAST when

- Tokenizer-only AR fix
- Keep discrete CE head
- Train-speed / compression
- Not high-Hz bimanual alone
- Different lever than OFT

Do NOT pitch OFT as “beats $\pi_{0.5}$ for BEHAVIOR” — best OpenVLA-family FT recipe only

RULE OF THUMB

OpenVLA / HF / LoRA ops + high Hz

→ OFT mandatory; stock AR obsolete for 25–50 Hz

Max wall-clock + OpenPI / JAX

→ π_0 / $\pi_{0.5}$ still win latency + pretrain coverage

Tokenizer-only AR fix

→ FAST (different lever; still AR latency)

Product line: best OpenVLA-family FT · competitive with π_0 on LIBERO + this ALOHA suite

RISKS · staying on OpenVLA family

Stakeholder honesty list

1 Fine-tuning 2024 OXE AR $\neq$ 2025 FM generalist

2 L1 unimodality — weak on true multimodal demos

3 FiLM tax — +456M params; eng complexity

4 ALOHA chunk latency 0.321 s vs π_0 0.086 s

5 “97% LIBERO” $\neq$ BEHAVIOR-ready

6 Put-in-pot $\sim$ 51% — not product-reliable

7 Dual-stack cost if also running $\pi_{0.5}$

8 FAST $\neq$ OFT confusion (different levers)

LIBERO Table I · SR detail

Method	Spatial	Object	Goal	Long	Avg
OpenVLA FT (vanilla)	84.7	88.4	79.2	53.7	76.5
+ PD&AC discrete	91.3	92.7	90.5	86.5	90.2
+ PD&AC Cont-Diff	96.9	98.1	95.5	91.1	95.4
OFT Cont-L1 (3rd)	96.2	98.3	96.2	90.7	95.3
OFT +wrist+proprio	97.6	98.4	97.9	94.5	97.1
π_0 FT	96.8	98.8	95.8	85.2	94.2

PD+AC $\approx$ +13.7 pp · continuous $\approx$ +5 pp · Long cell is the interesting win (94.5 vs π_0 85.2)

ALOHA board · completion scores

OFT+ vs π_0 — scoop 100 vs 93.3 · pot 51 vs 42

ACT	fold shorts / shirt / scoop / pot → 100 / 94 / 71 / 24
DP	fold shorts / shirt / scoop / pot → 100 / 100 / 79 / 31
RDT-1B	fold shorts / shirt / scoop / pot → 100 / 96 / 73 / 44
π_0 FT	fold shorts / shirt / scoop / pot → 100 / 100 / 93.3 / 42
OFT (no FiLM)	fold shorts / shirt / scoop / pot → — / — / 35 / 32
OFT+	fold shorts / shirt / scoop / pot → 100 / 100 / 100 / 51

Without FiLM → language $\approx$ chance (33%) on scoop/put · FiLM mandatory for multi-view

COST / GPU / DEMO BUDGET

FT GPUs

8× A100/H100 80GB · DDP + LoRA

LIBERO wall

1-2 days · 50-150K steps / suite

ALOHA demos

fold 20-30 · scoop 45 · pot 300

Trainable

279M LIBERO · 853M OFT+ (FiLM)

Scout budget

P0 <1 GPU-h · Partial 16-40

Infer

Single A100; 7B still heavier than ACT/DP/ π

CITE OFT FOR / DO NOT CITE FOR

Cite for

- ✓ FT recipe: parallel + chunk + Cont + L1
- ✓ Output signature shift vs vanilla
- ✓ High-Hz FT of large VLMs w/o LL controller
- ✓ Fair ablation morals vs Cont-Diff / PD&AC
- ✓ L1 $\approx$ diffusion SR at lower cost

Do not cite for

- ✗ Universal replacement for FM heads
- ✗ BEHAVIOR q-score SOTA
- ✗ Proof L1 models multimodal demos
- ✗ Fair ALOHA win without n/rubric/JAX notes
- ✗ Drop-in household champion

Fine-Tuning Vision-Language-Action Models: Optimizing Speed and Success

Moo Jin Kim¹ Chelsea Finn¹ Percy Liang¹

<https://openvla-oft.github.io>

Abstract—Recent vision-language-action models (VLAs) build upon pretrained vision-language models and leverage diverse robot datasets to demonstrate strong task execution, language following ability, and semantic generalization. Despite these successes, VLAs struggle with novel robot setups and require fine-tuning to achieve good performance, yet how to most effectively fine-tune them is unclear given many possible strategies. In this work, we study key VLA adaptation design choices such as different action decoding schemes, action representations, and learning objectives for fine-tuning, using OpenVLA as our representative base model. Our empirical analysis informs an Optimized Fine-Tuning (OFT) recipe that integrates parallel decoding, action chunking, a continuous action representation, and a simple L1 regression-based learning objective to altogether improve inference efficiency, policy performance, and flexibility in the model’s input-output specifications. We propose OpenVLA-OFT, an instantiation of this recipe, which sets a new state of the art on the LIBERO simulation benchmark, significantly boosting OpenVLA’s average success rate across four task suites from 76.5% to 97.1% while increasing action generation throughput by 26×. In real-world evaluations, our fine-tuning recipe enables OpenVLA to successfully execute dexterous, high-frequency control tasks on a bimanual ALOHA robot and outperform other VLAs (π_0 and RDT-1B) fine-tuned using their default recipes, as well as strong imitation learning policies trained from scratch (Diffusion Policy and ACT) by up to 15% (absolute) in average success rate. We release code for OFT and pretrained model checkpoints at <https://openvla-oft.github.io>.

I. INTRODUCTION

Recent vision-language-action models (VLAs)—robot policies built by fine-tuning pretrained vision-language models on large-scale robot datasets for low-level robotic control—have demonstrated strong task performance, semantic generalization, and language following abilities across diverse robots and tasks [4, 34, 24, 23, 7, 15, 8, 53, 58, 3]. Despite their strengths, fine-tuning is crucial for satisfactory deployment of VLAs on novel robots and tasks, yet it is unclear what the most effective approach for adaptation is given the large design space. A robotics practitioner who wishes to fine-tune a VLA to a new robot setup and task may default to using the same training recipe used for pretraining (or a parameter-efficient variant), but it is not obvious whether this would yield the best policy, and there is limited empirical analysis of alternative fine-tuning approaches in the literature.

¹Stanford University. Correspondence to: Moo Jin Kim <moojink@cs.stanford.edu>, Chelsea Finn <cbfinn@cs.stanford.edu>, Percy Liang <piliang@cs.stanford.edu>.

Prior work has begun exploring VLA adaptation strategies, with Kim et al. [23] proposing parameter-efficient fine-tuning via LoRA. However, their autoregressive action generation remains too slow (3-5 Hz) for high-frequency control (25-50+ Hz), and both LoRA and full fine-tuning of autoregressive VLAs often yield unsatisfactory performance in bimanual manipulation tasks [54, 27, 3]. While recent approaches improve efficiency through better action tokenization schemes [2, 38], achieving 2 to 13× speedups, significant latency between action chunks (e.g., 750 ms for the recent FAST approach [38]) still limits real-time deployment on high-frequency bimanual robots. Exploring alternative VLA adaptation approaches that achieve both satisfactory speed and quality remains an under-explored area of research.

In this work, we study key design decisions for adapting VLAs to novel robots and tasks using OpenVLA, a representative autoregressive VLA, as our base model. We examine three key design choices: action decoding scheme (autoregressive vs. parallel generation), action representation (discrete vs. continuous), and learning objective (next-token prediction vs. L1 regression vs. diffusion). Our study reveals several key insights that build on each other: (1) parallel decoding with action chunking not only boosts inference efficiency but also improves success rates on downstream tasks while enabling greater flexibility in the model’s input-output specifications; (2) continuous action representations further improve model quality compared to discrete representations; and (3) fine-tuning the VLA with an L1 regression objective yields comparable performance to diffusion-based fine-tuning while offering faster training convergence and inference speed.

Building on these insights, we introduce **OpenVLA-OFT**: an instantiation of an Optimized Fine-Tuning (OFT) recipe that integrates parallel decoding and action chunking, continuous action representations, and an L1 regression objective to enhance inference efficiency, task performance, and model input-output flexibility while maintaining algorithmic simplicity. We conduct experiments on both the standardized LIBERO simulation benchmark and dexterous tasks on a real bimanual ALOHA robot. In LIBERO, OpenVLA-OFT establishes a new state of the art by achieving 97.1% average success rate across four task suites, outperforming both fine-tuned OpenVLA policies [23] (76.5%) and π_0 policies [3] (94.2%) while achieving a 26× speedup in action generation with 8-step action chunks. For real-world ALOHA tasks [56], we augment our recipe with FiLM [37] for enhanced language grounding,

arXiv:2502.19645v2 [cs.RO] 28 Apr 2025

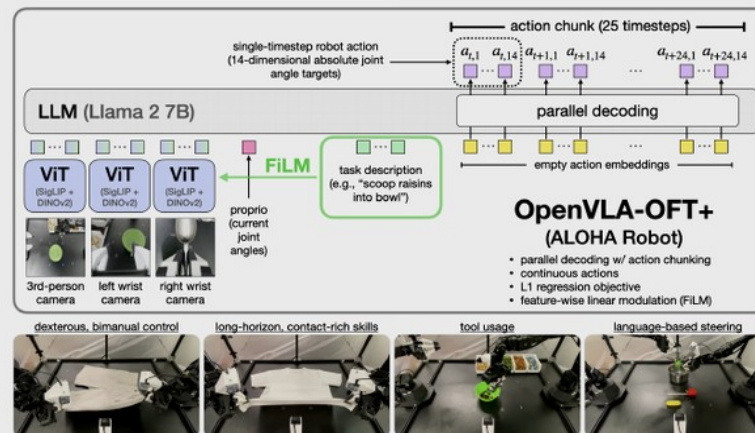


Fig. 1: OpenVLA-OFT+ on the bimanual ALOHA robot. Our Optimized Fine-Tuning (OFT) recipe enhances fine-tuned OpenVLA policies through improved inference efficiency, model quality, and input-output flexibility. The resulting OpenVLA-OFT+ policies execute diverse dexterous manipulation tasks on a real-world bimanual robot at high control frequencies (25 Hz). The “+” suffix indicates the integration of feature-wise linear modulation (FiLM) [37], which strengthens language grounding in tasks where accurate language understanding is critical for success.

denoting the augmented recipe as **OFT+**. OpenVLA-OFT+ successfully executes dexterous bimanual tasks like folding clothes and manipulating target food items based on the user’s prompt (see Figure 1), outperforming both fine-tuned VLAs (π_0 and RDT-1B) and prominent imitation learning policies trained from scratch (Diffusion Policy and ACT) by up to 15% (absolute) in average success rate. With 25-timestep action chunks, OpenVLA-OFT+ achieves $43\times$ faster throughput than base OpenVLA, demonstrating that our new fine-tuning recipe enables real-time robot control with strong task performance and language following ability.

II. RELATED WORK

Prior works have leveraged language and vision foundation models to enhance robotic capabilities, using them as pretrained visual representations that accelerate robotic policy learning [30, 33, 31, 20, 32], for object localization in robotics tasks [9, 47], and for high-level planning and reasoning [1, 17, 44, 16, 45, 18, 6]. More recently, researchers have explored fine-tuning vision-language models (VLMs) to directly predict low-level robotic control actions, producing “vision-language-action” models (VLAs) [4, 34, 24, 23, 7, 15, 8, 53, 58, 54, 3, 2], which have demonstrated effective language following [59] and generalization to out-of-distribution test conditions and unseen semantic concepts. These works focus primarily on model development, while we focus on developing a recipe for fine-tuning such models, justifying individual design decisions with insights that we gain from our empirical analysis.

Despite the importance of fine-tuning for real-world VLA deployment, empirical analysis of effective fine-tuning recipes remains limited. While Kim et al. [23] study various parameter update strategies and from their findings show that LoRA fine-tuning enables effective adaptation to single-arm robots operating at low control frequencies (< 10 Hz), their analysis does not extend to bimanual robots with high control frequencies (25-50+ Hz), a more complex control scenario. We address this gap by exploring VLA adaptation design decisions for fast inference and reliable task execution on a real-world bimanual manipulator with a 25 Hz controller.

Recent works by Belkhale and Sadigh [2] and Pertsch et al. [38] improve VLA efficiency through new action tokenization schemes, using vector quantization or discrete cosine transform-based compression to represent action chunks (sequences of actions) with fewer tokens than simple per-dimension binning (as used in RT-2 [4] and OpenVLA [23]). While these approaches achieve 2 to $13\times$ speedups for autoregressive VLAs, we explore design decisions beyond autoregressive modeling, which remains inherently limited by iterative generation. Our parallel decoding approach, when paired with action chunking, achieves significantly greater speedups: $26\times$ to $43\times$ throughput with much lower latency (0.07 ms for single-arm tasks with one input image and 0.321 ms for bimanual tasks with three input images).

Another line of research [54, 27, 3] demonstrates effective VLA fine-tuning for high-frequency, bimanual manipulation using generative approaches like diffusion or flow match-

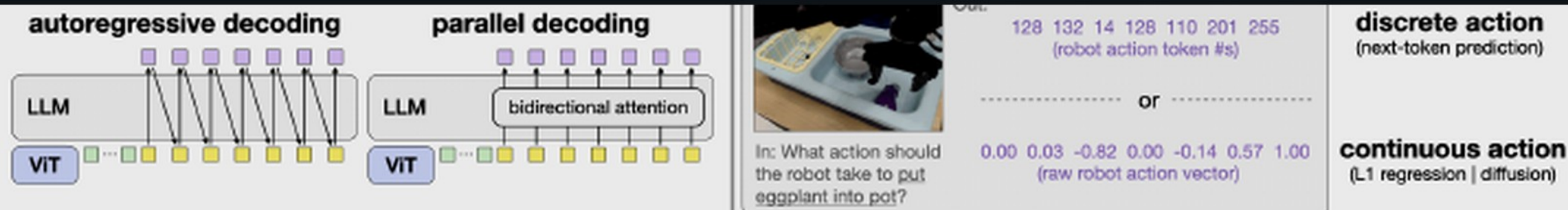


Fig. 2: Key design decisions for VLA fine-tuning. Left: Comparison between autoregressive decoding, which generates actions sequentially, and parallel decoding, which leverages bidirectional attention and generates all actions in a single forward pass. Right: Comparison between discrete action tokens with next-token prediction and continuous action values with L1 regression or diffusion modeling objectives. The original OpenVLA training scheme includes autoregressive decoding, discrete actions, and next-token prediction.

ing. While these diffusion-based VLAs achieve higher action throughput than autoregressive VLAs by generating multi-timestep action chunks simultaneously, they introduce computational trade-offs through slower training and multiple denoising or integration steps at inference time. Furthermore, these diffusion VLAs vary considerably in architecture, learning algorithm, vision-language fusion approach, and input-output specifications—and which design elements most significantly impact performance remains unclear. Through controlled experiments, we show that policies fine-tuned with a simpler L1 regression objective can match more complex approaches in

timestep action takes 0.33 seconds on an NVIDIA A100 GPU. For a chunk size of K timesteps and action dimensionality D , OpenVLA requires KD sequential decoder forward passes versus just D passes without chunking. This K -fold increase in latency makes action chunking impractical for high-frequency robots under the original formulation. In the next section, we present a parallel generation scheme that enables efficient action chunking.

IV. STUDYING KEY VLA FINE-TUNING DESIGN DECISIONS

Fig.1 crop · architecture + tasks

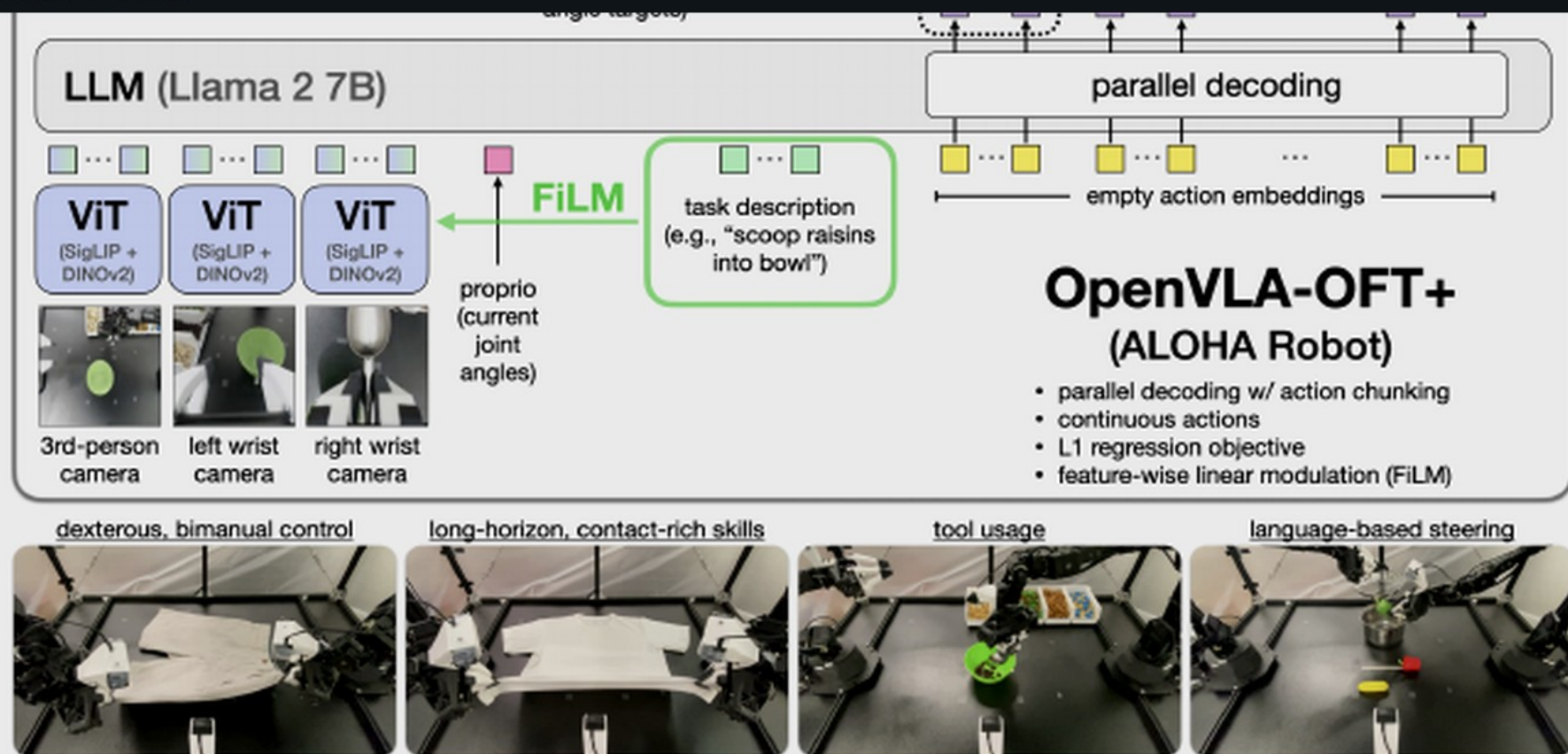


Fig. 1: OpenVLA-OFT+ on the bimanual ALOHA robot. Our Optimized Fine-Tuning (OFT) recipe enhances fine-tuned OpenVLA policies through improved inference efficiency, model quality, and input-output flexibility. The resulting OpenVLA-OFT+ policies execute diverse dexterous manipulation tasks on a real-world bimanual robot at high control frequencies (25 Hz). The "+" suffix indicates the integration of feature-wise linear modulation (FiLM) [37], which strengthens language grounding in tasks where accurate language understanding is critical for success.

denoting the augmented recipe as **OFT+**. OpenVLA-OFT+ successfully executes dexterous bimanual tasks like folding clothes and manipulating target food items based on the user's prompt (see Figure 1), outperforming both fine-tuned VLAs

Despite the importance of fine-tuning for real-world VLA deployment, empirical analysis of effective fine-tuning recipes remains limited. While Kim et al. [23] study various parameter update strategies and from their findings show that LoRA fine-

HARNESS vs BEHAVIOR DEPLOY

Visionary · 2026 — no Discord/Drive

HARNESS (keep OFT)

- Open AR-family control / ablation
- Parallel continuous chunk FT moral
- L1 specialist adaptation (fast)
- FiLM lesson for multi-cam lang
- Measure throughput AND latency
- Stock AR bins → museum piece

DEPLOY ($\pi_{0.5}$ / OpenPI)

- 2026 BEHAVIOR primary substrate
- FM(+RTC/KI) continuous chunks
- Stages > monolithic FT
- Multimodal / recovery heads
- Household open-world coverage
- OFT weights $\neq$ entrant

Adopt OFT morals, not OFT weights

Primary head likely FM ($\pi_{0.5}$ / N1.7), not 7B L1 MLP

Skip Discord/Drive logistics · LIBERO 97.1% $\neq$ BEHAVIOR q-score

Stakeholder · close

OFT modernizes OpenVLA ops @ 25-50 Hz — do not substitute for $\pi_{0.5}$ on BEHAVIOR.

Choose OFT: Prismatic/HF/LoRA · Choose $\pi_{0.5}$: OpenPI / long-horizon · Choose FAST: tokenizer-only AR

Q&A · Oct 26 · OpenVLA-OFT / 2502.19645

DECISION TREE · OFT vs $\pi_{0.5}$ vs FAST

Stakeholder HIGH priority — say this aloud

Need high-Hz control on OpenVLA stack?

NO

Stock OpenVLA OK for
<10 Hz language demos

YES → adopt OFT

parallel + chunk + Cont-L1

Choose OFT when

- Prismatic / HF / LoRA ops
- Simple L1 FT preferred
- LIBERO-style + moderate demos
- 7B AR-lineage ablations
- Wrist+proprio match robot I/O

Choose $\pi_{0.5}$ when

- OpenPI standardized
- Long-horizon open-world
- 2026 BEHAVIOR primary
- Need multimodal / recovery
- Max wall-clock (291 Hz class)

Choose FAST when

- Tokenizer-only AR fix
- Keep discrete CE head
- Train-speed / compression
- Not high-Hz bimanual alone
- Different lever than OFT

Do NOT pitch OFT as “beats $\pi_{0.5}$ for BEHAVIOR” — best OpenVLA-family FT recipe only

BOARD TABLE · Hz / SR

Memorize these cells — Stakeholder tattoo

LIBERO

SR avg	76.5% → 97.1%	+20.6 pp
Hz (A100)	4.2 → 109.7	26×
vs π_0 SR	π_0 94.2%	OFT wins
+wrist+proprio Hz	71.4	still $\gg$ vanilla

ALOHA (OFT+)

Hz	1.8 → 77.9	43×
π_0 Hz still	291.6	JAX faster
Scoop	100 vs π_0 93.3	OFT+ best
Put-in-pot	51 vs π_0 42	~half; hard

Latency (tables)

LIBERO Cont-L1 ~ 0.073 s
ALOHA OFT+ 0.321 s
(π_0 0.086 s)

Abstract caveat

“0.07 ms / 0.321 ms”
conflicts with Table II/III
Prefer seconds

Trainable

LIBERO OFT 279M
ALOHA OFT+ 853M
(FiLM +456M)

Rule: OpenVLA/HF/LoRA ops + high Hz → OFT mandatory; stock AR obsolete for 25-50 Hz

Max wall-clock + OpenPI/JAX → $\pi_0/\pi_{0.5}$ still win latency + pretrain coverage